```
import pandas as pd | import numpy as np | import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

**DECISION TREE**

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
df = pd.read_csv('data/carseats.csv')  |  df['High'] = (df['Sales'] > 8).astype(int)
df['ShelveLoc'] = pd.factorize(df['ShelveLoc'])[0] #this is how to factorize
df['Urban'] = df['Urban'].map({'No': 0, 'Yes': 1}) #this is turning categorical into numbers
df['US'] = df['US'].map({'No': 0, 'Yes': 1})
X = df.drop(['Sales', 'High'], axis=1)  |  y = df['High']
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8, random_state=0)
tree = DecisionTreeClassifier(max_depth=6, random_state=0)
tree.fit(X_train, y_train) | pred = tree.predict(X_test) | print(confusion_matrix(y_test, pred))
print('Test acc:', accuracy_score(y_test, pred)) | print(classification_report(y_test, pred))
```

**REGRESSION TREE**

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error |  df = pd.read_csv('data/boston.csv')
X = df.drop('medv', axis=1)  |  y = df['medv']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
tree = DecisionTreeRegressor(max_depth=4, random_state=1)
tree.fit(X_train, y_train) |  pred = tree.predict(X_test)
mse = mean_squared_error(y_test, pred)
print('Test MSE:', mse);  print('Test RMSE:', np.sqrt(mse))
```

**BAGGING, RANDOM FOREST** Bagging averages many bootstrap trees. Random forests is same but each split only use a random subset of predictors. this decorrelates trees & improves test performance.

```
from sklearn.ensemble import RandomForestRegressor  | p = X.shape[1] |
bag = RandomForestRegressor(n_estimators=500, max_features=p, random_state=1)
bag.fit(X_train, y_train)  | p_sqrt = int(np.sqrt(p))
rf = RandomForestRegressor(n_estimators=500, max_features=p_sqrt, random_state=1)
rf.fit(X_train, y_train)
print('Bagging MSE:', mean_squared_error(y_test, bag.predict(X_test)))
print('Random forest MSE:', mean_squared_error(y_test, rf.predict(X_test)))
importance = pd.Series(rf.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False).head())
```

**BOOSTING** Boosting grows trees sequentially. Each new tree improves the current model, by fitting residuals. Small learn rate means slower learning(more trees), large learn rate can overfit.

```
from sklearn.ensemble import GradientBoostingRegressor
for lr in [0.01, 0.2]: model = GradientBoostingRegressor(
    n_estimators=500, learning_rate=lr, max_depth=4,  random_state=1  )
  model.fit(X_train, y_train)  |  pred = model.predict(X_test)
  print(f'Learning rate {lr}, test MSE:', mean_squared_error(y_test, pred))
```

**SVM** C controls the cost of margin violations. Small C= wider margin and more support vectors. Large C gives a narrower margin and penalizes violations more.

```
from sklearn.svm import SVC | from sklearn.model_selection import train_test_split,
GridSearchCV  | from sklearn.metrics import confusion_matrix, accuracy_score
np.random.seed(5)  |  X = np.random.randn(20, 2)
y = np.repeat([1, -1], 10)  |  X[y == -1] = X[y == -1] + 2
params = {'C': [0.001, 0.01, 0.1, 1, 5, 10, 100]}
grid = GridSearchCV(SVC(kernel='linear'), params, cv=10, scoring='accuracy')
grid.fit(X, y)  |  print('Best C:', grid.best_params_)print('Best CV score:', grid.best_score_)
svc = grid.best_estimator_  |  pred = svc.predict(X)
print(confusion_matrix(y, pred))  |  print('Training accuracy:', accuracy_score(y, pred))
print('Number of support vectors:', svc.support_.size)
```

**radial kernel SVM** radial kernel creates nonlinear boundaries. gamma controls how local the influence of each point is. High gamma can make the boundary very flexible and may overfit.

```
np.random.seed(1)  |  X = np.random.randn(200, 2)
X[:100] = X[:100] + 2  |  X[100:150] = X[100:150] – 2 | y = np.repeat([1, -1], 100)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=1)
params = {'C': [0.1, 1, 10, 100], 'gamma': [0.5, 1, 2, 3]}
grid = GridSearchCV(SVC(kernel='rbf'), params, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)  |  print('Best parameters:', grid.best_params_)
pred = grid.best_estimator_.predict(X_test)
print(confusion_matrix(y_test, pred))  |  print('Test accuracy:', accuracy_score(y_test, pred))
```

| Kernel | Formula Idea | Best use |
|---|---|---|
| Linear | kernel = 'linear' | Roughly Linear Boundary |
| Polynomial | kernel = 'poly', degree=3 | Curved Polynomial boundary |
| Radial/RBF | kernel = 'rbf' | Flexible non-linear boundary |

**ROC curve for SVM**

```
from sklearn.metrics import roc_curve, auc  # ROC curve for SVM
for gamma in [0.5, 3]:
  model = SVC(kernel='rbf', gamma=gamma, C=1)
  model.fit(X_train, y_train)  |  scores = model.decision_function(X_test)
  fpr, tpr, threshold = roc_curve(y_test, scores) |  roc_auc = auc(fpr, tpr)
  plt.plot(fpr, tpr, label=f'gamma={gamma}, AUC={roc_auc:.3f}')
plt.plot([0, 1], [0, 1], 'k--') | plt.xlabel('False Positive Rate')  #outside of loop
plt.ylabel('True Positive Rate') |  plt.legend() |  plt.show()
```

**PCA** PCA finds directions of maximum variance. Scaling is important because variables have different units and variances. Loadings are variable weights; scores are observation coordinates.

```
from sklearn.preprocessing import scale | from sklearn.decomposition import PCA
df = pd.read_csv('data/USArrests.csv', index_col=0)
X = pd.DataFrame(scale(df), index=df.index, columns=df.columns)
pca = PCA()  |  scores = pca.fit_transform(X)
loadings = pd.DataFrame( pca.components_.T,  index=df.columns,
   columns=[f'PC{i}' for i in range(1, X.shape[1] + 1)] )
print(loadings)print('PVE:', pca.explained_variance_ratio_)
print('Cumulative PVE:', np.cumsum(pca.explained_variance_ratio_))
plt.plot(range(1, 5), pca.explained_variance_ratio_, '-o')
plt.xlabel('Principal Component') | plt.ylabel('Proportion of Variance Explained')  | plt.show()
X_raw = pd.read_csv('data/nci60_data.csv', index_col=0) # example2 (labs.csv hidden)
X = scale(X_raw) |  pca = PCA()  |  scores = pca.fit_transform(X)
color_id = pd.factorize(labs.iloc[:, 0])[0]
plt.scatter(scores[:, 0], scores[:, 1], c=color_id, cmap='hsv', alpha=0.7)
plt.xlabel('PC1')  |  plt.ylabel('PC2')  |  plt.title('NCI60 PCA')  |  plt.show()
print(f"{'Component':<12} | {'Variance':<20} | {'Cumulative Variance':<20}")  |  print("-" * 59)
for i, (var, cum_var) in enumerate(zip(pca.explained_variance_ratio_[:10],
np.cumsum(pca.explained_variance_ratio_)[:10]), 1):
  print(f"PC{i:<10} | {var:<20.2%} | {cum_var:<20.2%}")
```

**Hierarchical** Single linkage can create chaining; complete and average linkage

```
from scipy.cluster.hierarchy import linkage, dendrogram, cut_tree | np.random.seed(123)
X = np.random.randn(50, 2) | X[:25, 0] = X[:25, 0] + 3X[:25, 1] = X[:25, 1] – 4
for method in ['complete', 'average', 'single']:
  hc = linkage(X, method=method) |  plt.figure(figsize=(10, 5))
  dendrogram(hc, leaf_rotation=90, leaf_font_size=8)
  plt.title(f'{method} linkage') | plt.xlabel('Observation') | plt.ylabel('Distance') | plt.show()
hc_complete = linkage(X, method='complete')
clusters = cut_tree(hc_complete, n_clusters=2).reshape(-1)print(clusters)
specific_observations = [0, 24, 25, 49] #use this find specific obs cluster
for idx in specific_observations: print(f"Observation {idx} -> Cluster {clusters[idx]}"
print("Observations in Cluster 0:", np.where(clusters == 0)[0]) #print all obs in cluster
```

**MLP** Neural networks should use scaled inputs. Hidden layers allow nonlinear decision boundaries.

```
from sklearn import preprocessing | from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, classification_report
url = 'https://…'  |  names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'Class']
iris = pd.read_csv(url, names=names) | X = iris.iloc[:, 0:4]  |  y = iris[['Class']]
y = y.apply(preprocessing.LabelEncoder().fit_transform)
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=1)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  |  X_test = scaler.transform(X_test)
mlp = MLPClassifier(hidden_layer_sizes=(10, 10, 10), max_iter=1000, random_state=1)
mlp.fit(X_train, y_train.values.ravel())
pred = mlp.predict(X_test)  |  print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

**MLP regressor** Ridge is linear, so it struggles with nonlinear patterns. An MLP with hidden layers can learn nonlinear relationships. ReLU gives piecewise linear fits; tanh/logistic activations are smoother.

```
from sklearn.linear_model import Ridge|from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score
np.random.seed(0) | N = 2000 | X = 0.5 * np.random.normal(size=N) + 0.35
Xt = 0.75 * X - 0.35  |  X = X.reshape((N,1))
y = -(8 * Xt**2 + 0.1 * Xt + 0.1) + 0.05 * np.random.normal(size=N)
y = np.exp(y) + 0.05*np.random.normal(size=N)  |  y = y / max(abs(y))
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)
ridge = Ridge().fit(X_train, y_train)  |  ridge_pred = ridge.predict(X_test)
mlp = MLPRegressor(hidden_layer_sizes=(16, 8), activation='tanh', max_iter=1000,
random_state=0)  |  mlp.fit(X_train, y_train)  |  mlp_pred = mlp.predict(X_test)
print('MLP MSE:', mean_squared_error(y_test, mlp_pred)) #change mlp to ridge if need
print('MLP R2:', r2_score(y_test, mlp_pred))  #change mlp to ridge if need
plt.scatter(X_test[:, 0], y_test, s=10, alpha=0.4, label='True')
plt.scatter(X_test[:, 0], mlp_pred, s=10, alpha=0.4, label='MLP') | plt.legend() | plt.show()
```

| Problem | Hidden activation | Output activation |
|---|---|---|
| Binary classification | ReLU or tanh | Sigmoid |
| Multi classification | | Softmax |
| Regression | | Linear |

**K-means** need K for fitting. Inertia is within-cluster sumofsquares. It reduce as K increase

```
from sklearn.cluster import KMeans | np.random.seed(123)  | X = np.random.randn(50, 2)
X[:25, 0] = X[:25, 0] + 3  |  X[:25, 1] = X[:25, 1] - 4
inertias = [] | K_range = range(1, 8)  #ELBOW PLOT
for k in K_range:
  km = KMeans(n_clusters=k, random_state=42, n_init='auto')
  km.fit(X) |  inertias.append(km.inertia_)  # Total within-cluster sum of squares
plt.figure(figsize=(12, 5)) |  plt.subplot(1, 2, 1)
plt.plot(K_range, inertias, marker='o', linestyle='--', color='b')
plt.xlabel('Number of Clusters (k)')  |  plt.ylabel('Inertia (Within-Cluster Cost)')
plt.title('Elbow Method For Optimal k') | plt.grid(True)
for k in [2, 3]:
```

```
km = KMeans(n_clusters=k, n_init=20, random_state=123)
km.fit(X) | print(f'K={k}') | print('Inertia:', km.inertia_)
print('Centers:\n', km.cluster_centers_)
plt.scatter(X[:, 0], X[:, 1], c=km.labels_, cmap='prism')
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1], marker='*', s=200,
    color='black') | plt.title(f'K-means with K={k}') | plt.show()
```

**K-mode** need K for fitting. Inertia is within-cluster sumofsquares. It reduce as K increase

```
from kmodes.kmodes import KModes | np.random.seed(123)
cat1 = np.random.choice(['A', 'B', 'C'], size=50) | cat2 = np.random.choice(['X', 'Y'], size=50)
df_cat = pd.DataFrame({'Cat1': cat1, 'Cat2': cat2})
```

```
costs = [] | K_range = range(1, 6) #ELBOW PLOT
for k in K_range: # cost is Total sum of categorical mismatches
    km = KModes(n_clusters=k, init='Cao', n_init=5, random_state=42)
    km.fit(df_cat) | costs.append(km.cost_)
plt.plot(K_range, costs, marker='o', linestyle='--', color='b')
plt.xlabel('Number of Clusters (k)') | plt.ylabel('Cost (Categorical Mismatches)')
plt.title('K-Modes Elbow Method') | plt.grid(True)
```

```
for k in [2, 3]:
    km = KModes(n_clusters=k, init='Cao', n_init=5, random_state=123)
    labels = km.fit_predict(df_cat) | print(f'K={k}')
    print('Cost:', km.cost_) | print('Centroids (Modes):\n', km.cluster_centroids_)
    plt.scatter(
        df_cat.iloc[:, 0].astype('category').cat.codes + np.random.uniform(-0.1, 0.1, 50),
        df_cat.iloc[:, 1].astype('category').cat.codes + np.random.uniform(-0.1, 0.1, 50),
        c=labels, cmap='prism') | plt.title(f'K-Modes with K={k}') | plt.show()
```

**K-prototype** for both categorical and numeric

```
from kmodes.kprototypes import KPrototypes
num_data = np.random.randn(50, 2) | num_data[:25, 0] += 3 | num_data[:25, 1] -= 4
cat_data = np.random.choice(['High', 'Low'], size=50).reshape(-1, 1)
df_mixed = pd.DataFrame(np.hstack([num_data, cat_data]), columns=['Num1', 'Num2',
'Cat1']) | df_mixed[['Num1', 'Num2']] = df_mixed[['Num1', 'Num2']].apply(pd.to_numeric)
cat_indices = [2] # Index of the categorical column
costs = [] | K_range = range(1, 6) # ELBOW PLOT
for k in K_range: # Combined Euclidean + Category mismatch cost
    kp = KPrototypes(n_clusters=k, init='Cao', n_init=5, random_state=42)
    kp.fit(df_mixed, categorical=cat_indices) | costs.append(kp.cost_)
plt.figure(figsize=(12, 5)) | plt.subplot(1, 2, 1)
plt.plot(K_range, costs, marker='o', linestyle='--', color='b')
plt.xlabel('Num. of Clusters (k)') | plt.ylabel('Cost') | plt.title('K-Proto Elbow Method') |
plt.grid(True) | for k in [2, 3]:
    kp = KPrototypes(n_clusters=k, init='Cao', n_init=5, random_state=123)
    labels = kp.fit_predict(df_mixed, categorical=cat_indices)
    print(f'K={k}') | print('Cost:', kp.cost_)
    print('Centroids (Means/Modes):\n', kp.cluster_centroids_)
    plt.scatter(df_mixed.iloc[:, 0], df_mixed.iloc[:, 1], c=labels, cmap='prism')
    plt.scatter(kp.cluster_centroids_[:, 0].astype(float), kp.cluster_centroids_[:,
      1].astype(float), marker='*', s=200, color='black')
    plt.title(f'K-Prototypes with K={k}') | plt.show()
```

K-means (manual)

| I | x1 | x2 | x3 | Grp |
|---|---|---|---|---|
| 1 | 10 | 2 | -1 | A |
| 2 | 11 | 3 | -1 | B |
| 3 | 18 | 5 | -1 | A |
| 4 | 20 | 4 | 0 | B |
| 5 | 19 | 3 | 0 | A |
| 6 | 8 | 2 | -1 | B |

x1 = (10 + 18 + 19) / 3 = 15.667 **step 1**
x2 = (2 + 5 + 3) / 3 = 3.333 calc centroid
x3 = (-1 + -1 + 0) / 3 = -0.667
CA = (15.667, 3.333, -0.667)

d(Oi, CA) = sqrt((x1i - x1CA)^2 + (x2i - x2CA)^2 + (x3i - x3CA)^2) **step2** dist to CA, reassign based on closest
**step3** if change, recalculate centroid
**step4** recalc dist

K-modes (manual)

| Obj | x1 | x2 | x3 | Grp |
|---|---|---|---|---|
| 1 | A | L | M | C1 |
| 2 | B | P | C | C2 |
| 3 | A | L | C | C1 |
| 4 | B | L | C | C1 |
| 5 | A | P | M | C2 |
| | | | | |

C1(A,L,C) C2(B,P,M)
**step 1** calc centroid
D(O1,C1) = x1: A vs A = 0
x2: L vs L = 0 | x3: M vs C = 1
dissimilarity = 0 + 0 + 1 = 1
D(O1,C2) = 1 + 1 + 0 = 2
**step2** Assign each obj to a centroid
**step3** Recalc centroid based on obj in it
**step4** if C change, recalc dissimilarity

K medoids
**step 1** init medoid
**step 2** assign each point to medoid based on dist
**step 3** Calculate total cost (every point to its assigned medoid
**step 4** Calculate cost to a new medoid, if lower than swap

**Cross Validation** repeats train/test split by cv = 10 times.

```
from sklearn.linear_model import LinearRegression | from sklearn.preprocessing import
PolynomialFeatures | from sklearn.model_selection import cross_val_score, KFold
df = pd.read_csv('data/auto.csv', na_values='?').dropna()
X = df[['horsepower']].values | y = df['mpg'].values
lr = LinearRegression()for label, cv in [('LOOCV', KFold(n_splits=len(X))),
        ('10-Fold', KFold(n_splits=10, shuffle=True, random_state=1))]:
    print(f"\n--- {label} ---")
    for degree in [1, 2]:
        poly = PolynomialFeatures(degree=degree)
        X_p = poly.fit_transform(X)
        scores = cross_val_score(lr, X_p, y, cv=cv, scoring='neg_mean_squared_error')
        print(f"  Degree {degree}: MSE = {-scores.mean():.2f}")
```

**Bootstrap** logistic regression with bootstrap. Sampling with replacement. ~36% will not get picked.

```
import statsmodels.formula.api as smf
df = pd.read_csv('data/default.csv')
df['default_yes'] = (df['default'] == 'Yes').astype(int) | params_list = []
for i in range(100):
    sample = df.sample(len(df), replace=True)
    result = smf.logit('default_yes ~ balance + income', data=sample).fit(disp=0)
    params_list.append(result.params)
boot_df = pd.DataFrame(params_list)print("Bootstrap Standard Errors:")print(boot_df.std())
result_full = smf.logit('default_yes ~ balance + income', data=df).fit()print("\nModel Standard
Errors:") | print(result_full.bse)
```

**KNN** Supervised. Already know red or blue, grey point comes in assign to average of closest k neighbours.

```
import pandas as pdimport numpy as npfrom sklearn.neighbors import
KNeighborsRegressorfrom sklearn.model_selection import train_test_split,
GridSearchCVfrom sklearn.metrics import mean_squared_errorfrom math import sqrt
df = pd.read_csv('data/wage.csv') | X = df[['age']] | y = df['wage']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)# Grid search for best Kmodel = GridSearchCV(
    KNeighborsRegressor(),
    param_grid={"n_neighbors": list(range(1, 31))}, cv=5,
    scoring='neg_mean_squared_error')
model.fit(X_train, y_train)
best_k = model.best_params_['n_neighbors']print("Best K:", best_k)
y_pred = model.predict(X_test)
rmse = sqrt(mean_squared_error(y_test, y_pred))print(f"Test RMSE: {rmse:.2f}")
```

**Ridge** shrinks coefficients toward zero (L2 penalty: λΣβ²) but never exactly to zero, all features are kept. StandardScaler ensures same scale before penalisation. >alpha = stronger shrinkage = simpler model.

```
from sklearn.linear_model import Ridge, RidgeCVfrom sklearn.preprocessing import
StandardScalerfrom sklearn.model_selection import train_test_splitfrom sklearn.metrics
import mean_squared_error
df = pd.read_csv('data/hitters.csv').dropna().drop('Unnamed: 0', axis=1)
y = df['Salary'].values
X = df.drop(['Salary', 'League', 'Division', 'NewLeague'], axis=1).astype('float64')
scaler = StandardScaler() | X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
random_state=42)
rcv = RidgeCV(alphas=np.linspace(0.01, 100, 1000), cv=10)
rcv.fit(X_train, y_train) | print("Best alpha:", rcv.alpha_)
ridge = Ridge(alpha=rcv.alpha_) # Refit Ridge with best alpha and evaluate
ridge.fit(X_train, y_train)
mse = mean_squared_error(y_test, ridge.predict(X_test)) | print(f"Test MSE: {mse:.2f}")
coef_series = pd.Series(ridge.coef_, index=X.columns).abs().sort_values(ascending=False)
print("Top 3 predictors:\n", coef_series.head(3))
```

**Lasso** Lasso uses L1 penalty (λΣ|β|) which shrink coefficients to zero, performing feature selection. This makes Lasso preferable over Ridge when you suspect only a few predictors truly matter.

```
from sklearn.linear_model import Lasso, LassoCV | from sklearn.preprocessing import
StandardScaler | from sklearn.model_selection import train_test_split | from
sklearn.metrics import mean_squared_error
df = pd.read_csv('data/hitters.csv').dropna().drop('Unnamed: 0', axis=1)
y = df['Salary'].values | X = df.drop(['Salary', 'League', 'Division', 'NewLeague'],
axis=1).astype('float64') | scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
random_state=42)# LassoCV to find best alpha
lasso_cv = LassoCV(alphas=np.linspace(0.01, 100, 1000), cv=10, max_iter=100000)
lasso_cv.fit(X_train, y_train) | print("Best alpha:", lasso_cv.alpha_)
lasso = Lasso(alpha=lasso_cv.alpha_, max_iter=100000) | lasso.fit(X_train, y_train)
mse = mean_squared_error(y_test, lasso.predict(X_test)) | print(f"Test MSE: {mse:.2f}")
coef_series = pd.Series(lasso.coef_, index=X.columns)
print(f"\nCoefficients set to zero: {(coef_series == 0).sum()}")
print("Non-zero features:\n", coef_series[coef_series != 0])
```